PROJECT OVERVIEW · HOME NAS

Turning an old gaming laptop into a home NAS

How four spare 1TB drives and a retired MSI laptop became one protected, shared photo server, with a Google Photos-style app and secure access from anywhere. Planned, built and troubleshot from October 24 to November 1, 2026.

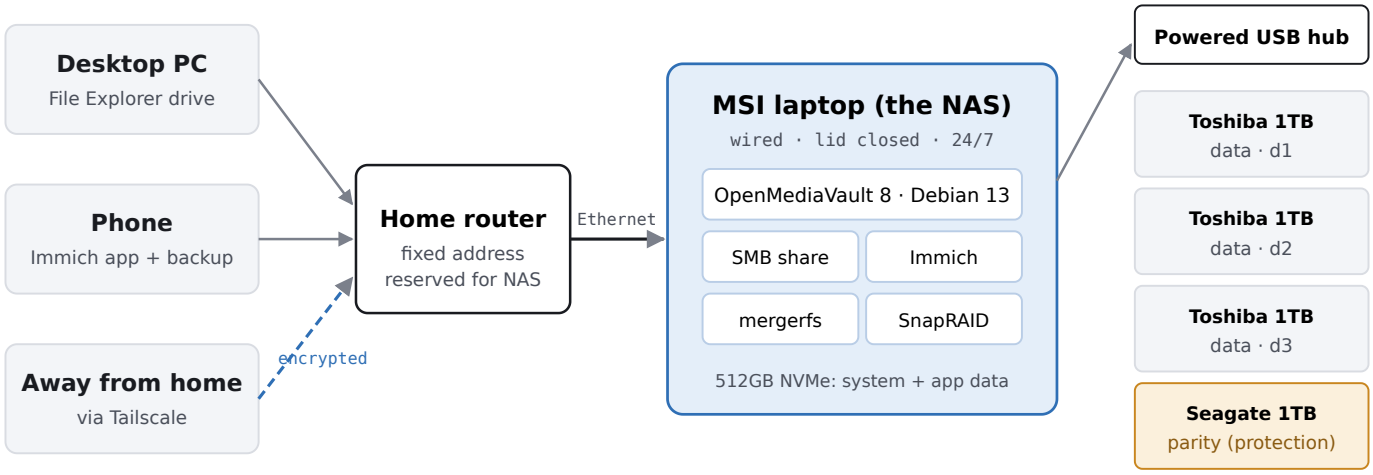


Fig. 1 · The finished system. Every device reaches the NAS through the home router. Away from home, Tailscale provides an encrypted private link, so no router ports are open.

~2.7 TB
USABLE, PROTECTED STORAGE

10,115
PHOTO & VIDEO FILES MOVED, 0 LOST

$258
OF PI PARTS NEVER BOUGHT

Dennis Cochardo Built Oct 24 – Nov 1, 2026

Network addresses, account names and passwords are intentionally left out.

01 · SUMMARY

What was built, in one page

Four 1TB portable hard drives were sitting unused, and the photos on one of them existed nowhere else. The goal was a single shared drive the whole house could use, with protection against a failed drive, a proper photo app, and safe access from anywhere, built mostly from hardware already on hand.

The original plan used a Raspberry Pi 5. When memory-driven price increases pushed the 8GB board to about $200, the build moved to an old MSI gaming laptop instead. It runs OpenMediaVault, a free NAS operating system built on Debian Linux, with the lid closed around the clock.

The process

#	STAGE	WHAT HAPPENED	DATE
01	Plan	Chose OpenMediaVault, mergerfs and a powered USB hub; priced a Raspberry Pi 5 build at about $307.	Oct 24–27
02	Pivot	Switched to an old MSI GF65 Thin laptop and matched a replacement 180W charger.	Oct 30–31
03	Install	Wrote the installer USB with Rufus (DD mode), booted it from the F11 menu, installed OpenMediaVault 8.3.1 on the internal NVMe drive.	Oct 31
04	Harden for 24/7	Lid switch ignored, sleep disabled, CPU on powersave, fixed address reserved on the router, updates applied.	Oct 31
05	Protect the photos	Copied ~146 GB of photos to a PC and verified every file with robocopy before touching the drives.	Oct 31
06	Prepare drives	Wiped all four drives and formatted them as ext4.	Oct 31–Nov 1
07	Pool & share	Pooled the drives with mergerfs, created a user and SMB share, mapped it in Windows and copied the photos back.	Nov 1
08	Photo app	Ran Immich in Docker, pointed at the photo library read-only.	Nov 1
09	Redundancy	Emptied one drive, made it the SnapRAID parity drive, ran the first sync and scheduled nightly protection.	Nov 1
10	Remote access	Added Tailscale for encrypted access from anywhere, with no open ports.	Nov 1

Contents

02 Timeline
03 Hardware and the laptop decision
04 The software stack
05 Installing OpenMediaVault
06 Running 24/7 on a laptop
07 Moving the photos safely
08 Storage design: mergerfs + SnapRAID
09 Sharing on the network
10 Immich, the photo app
11 Remote access with Tailscale
12 Troubleshooting log
13 Running costs
14 Lessons and next steps

02 · TIMELINE

Nine days, from parts list to protected server

Most of the hands-on work happened over two days. The long waits were drive formatting overnight and the first parity sync.

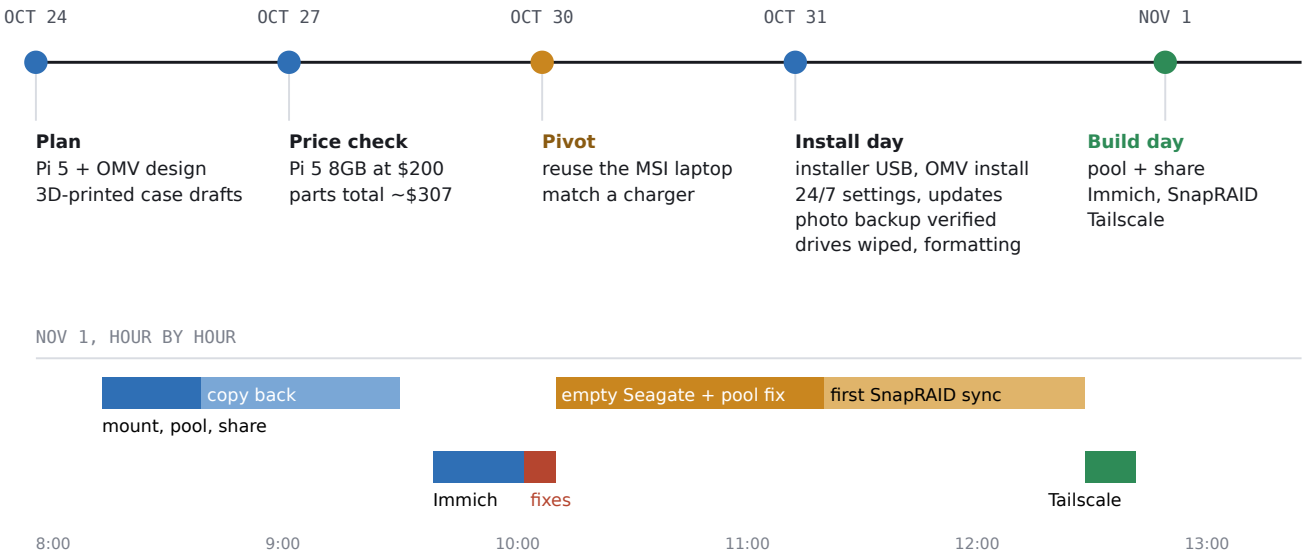


Fig. 2 · Project timeline. The orange block on November 1 includes the pool loop fix described in the troubleshooting log.

Hardware and the laptop decision

The Raspberry Pi plan

The first design was a Raspberry Pi 5 running OpenMediaVault, with the four drives on a powered USB 3 hub and a custom 3D-printed enclosure (three design iterations in OpenSCAD). When the parts were priced on October 27, the 8GB Pi alone had risen to $200 because of memory-driven price increases, from an $80 launch price.

RASPBERRY PI 5 BUILD, PARTS PRICED OCT 27, 2026 (USD)

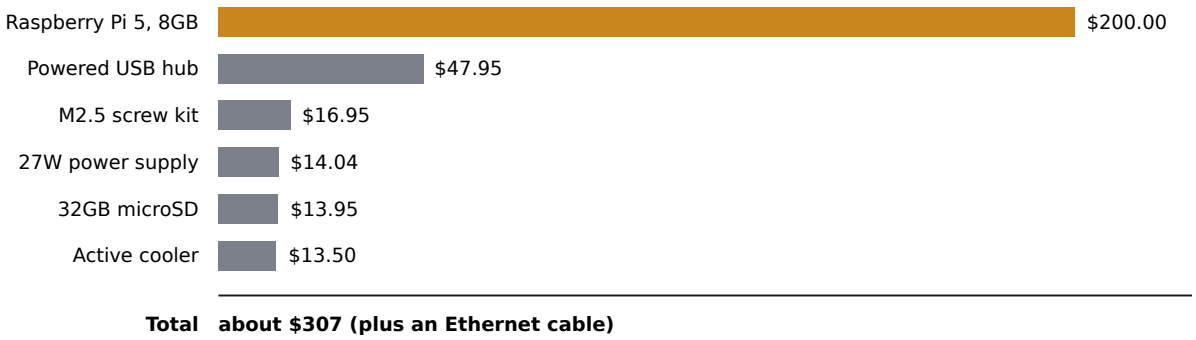


Fig. 3 · The Pi board alone was two-thirds of the cost. Reusing the laptop removed every part except the hub, a cable and a USB stick.

The laptop that replaced it

PART	DETAILS	ROLE
Laptop	MSI GF65 Thin 10UE · Intel Core i7 (10th gen) · NVIDIA RTX 3060 · Gigabit Ethernet	The NAS itself
System drive	Micron 2210 512GB NVMe (internal)	OpenMediaVault, Docker, Immich database
Charger	Insignia 180W universal, 5.5 × 2.5 mm tip (original spec 20V, 9A, 180W)	Always-on power
Data drives	3 × Toshiba MQ04UBF100 1TB (Canvio Advance)	File storage, pooled
Parity drive	1 × Seagate Portable 1TB	SnapRAID protection
Hub & network	Powered USB hub · Ethernet to the router	Powers drives · wired connection
Installer	32GB SanDisk USB stick	One-time OS installer

> A 10th-gen i7 is far more than a file server needs, and that headroom is what makes running Immich's face and object recognition on the same machine practical.

Charger note. The original charger was missing. A universal 180W charger with the 5.5 × 2.5 mm barrel tip from its compatibility guide works. A 19.5V or 20V charger is interchangeable here. What matters is the plug size, at least 180W, and a reputable brand.

04 · SOFTWARE STACK

The software stack, layer by layer

OpenMediaVault is not an app you run inside Windows. It replaces the operating system: a Debian Linux base with a web dashboard and NAS services on top. Add-ons come from omv-extras, a community plugin catalog. Apps like Immich run in Docker containers, each sealed off from the rest of the system.

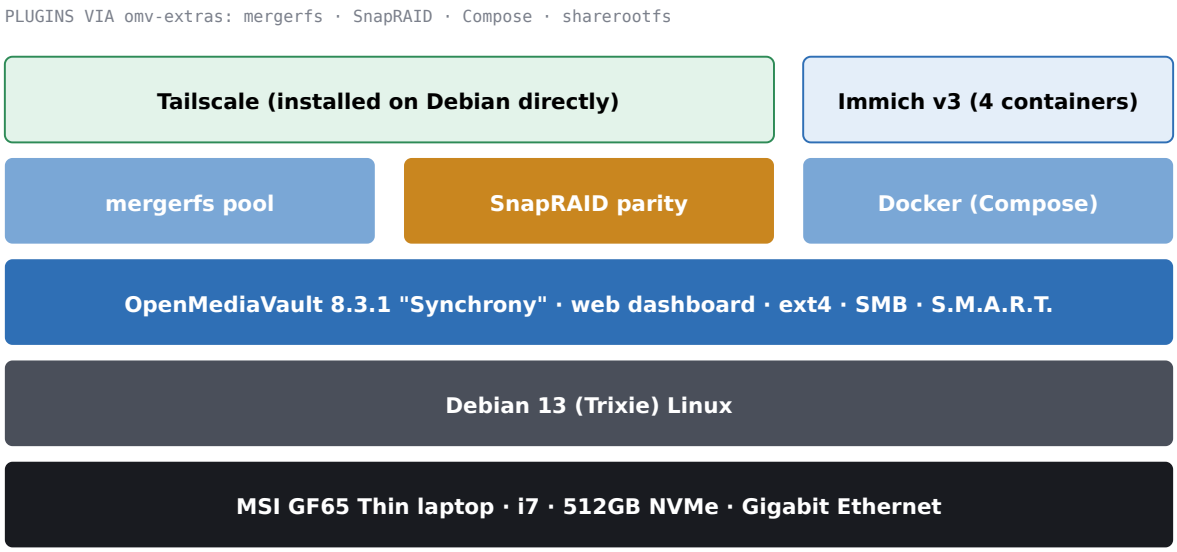


Fig. 4 · Each layer depends only on the one below it. The sharerootfs plugin lets app data live on the fast internal drive rather than the USB pool.

COMPONENT	JOB	WINDOWS EQUIVALENT
Debian 13	Base operating system	Windows itself
OpenMediaVault	Dashboard for drives, users, shares, updates	Settings + Control Panel, as a web page
SMB/CIFS	Shares the pool with PCs and phones	Right-click > Share
mergerfs	Makes three drives look like one	Storage Spaces
SnapRAID	Can rebuild any one failed drive	Storage Spaces parity
Docker Compose	Runs apps in isolated containers	Something like lightweight virtual machines
Immich	Photo timeline, albums, search, phone backup	Google Photos, self-hosted
Tailscale	Private encrypted network between your devices	A personal VPN

Installing OpenMediaVault

The installer is an ISO, a single file that holds a complete disc image. Copying it onto a USB stick doesn't make the stick bootable, because a computer reads startup instructions from the very beginning of a drive, not from files on it. Rufus writes the image onto the stick byte for byte (DD mode), so the stick becomes a clone of the installer disc.



Fig. 5 · Install path. Everything after step 5 was done from a desktop browser and an SSH session in PowerShell. The laptop never needed a keyboard again.

Details that mattered

- **External drives stayed unplugged** during the install, so the installer could only see the laptop's internal drive. The installer erases whatever disk you pick.
- **Ethernet was connected first.** The installer configures networking and pulls updates during setup. Without a cable, the network has to be fixed afterward from a text menu (`omv-firstaid`).
- **The console banner showed two addresses.** One was leftover text baked into the installer image by whoever built it. The real one matched the home network's range.
- **First login** used OpenMediaVault's default web account, whose password was changed right away, then all updates were installed. The connection dropping mid-update is normal, because the dashboard restarts itself.
- **A reboot** was required afterward to load a new Linux kernel.

06 · ALWAYS ON

Running 24/7 on a laptop

A laptop is designed to sleep, throttle and live on battery. A server needs the opposite. Five settings turn it into a quiet, always-on appliance.

SETTING	HOW	WHY
Ignore the lid	A `logind.conf.d` drop-in: HandleLidSwitch, …ExternalPower and …Docked all set to `ignore`	The lid can be closed without the NAS going offline
Never sleep	`systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target`	Nothing can put it to sleep, even by accident
CPU scaling: powersave	System > Power Management	Idles low, speeds up under load (see below)
Power button: Power Off	System > Power Management	A button press shuts down cleanly
Fixed network address	DHCP reservation on the router; the NAS stays on DHCP	Bookmarks, mapped drives and apps never break; no risk of a static-IP typo locking you out

```
# Lid and sleep settings, run once over SSH
mkdir -p /etc/systemd/logind.conf.d
printf
'[Login]\nHandleLidSwitch=ignore\nHandleLidSwitchExternalPower=ignore\nHandleLidSwitchDocked=ignore\n' \
  > /etc/systemd/logind.conf.d/nas-lid.conf
systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target
systemctl restart systemd-logind
```

Why "powersave" isn't slow here

The dashboard described three CPU options but only offered two. This i7 uses Intel's own frequency driver, `intel_pstate`, which only exposes *powersave* and *performance*. Under that driver, powersave still scales with load, the way *schedutil* does on other machines. Performance would pin the CPU high all day for no benefit.

The battery: a known limitation

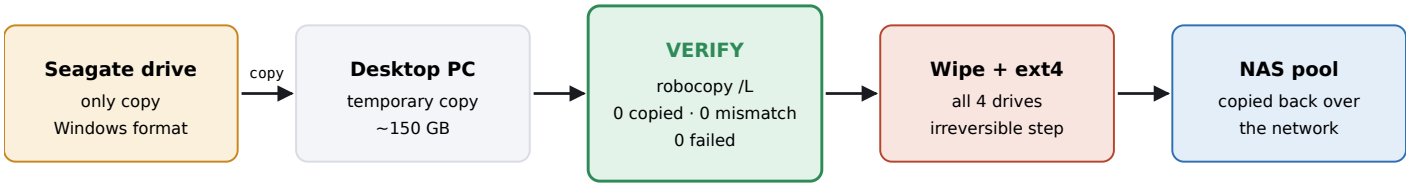
A laptop held at 100% charge forever wears its battery and can make it swell. MSI's Battery Master charge limit wasn't available on this model, and Linux's `msi-ec` driver returned "operation not supported" for its firmware. The mitigation is a visual check every few months: a bulging bottom panel, a lifting keyboard or a strange trackpad click means the battery should come out.

> Upside of a laptop: the battery is a built-in backup power supply. A brief outage doesn't cut power to the drives mid-write.

07 · DATA SAFETY

Moving the photos without losing one

The Seagate drive held the only copy of years of photos and GoPro footage. Formatting it for Linux would erase it, so nothing was wiped until a verified copy existed elsewhere.



The wipe happened only after verification passed. The PC copy was later deleted, which is what made adding SnapRAID parity a priority.

Fig. 6 · The rule was verify, then wipe. Every arrow before the red box is reversible; the red box is not.

Verification results

FOLDER	FILES	SIZE	MISSING	MISMATCHED	FAILED
Photos	4,867	78.6 GB	0	0	0
Pic backup after Navy	5,202	67.6 GB	0	0	0
GoPro footage	46	4.2 GB	0	0	0
Total	**10,115**	**~150 GB**	**0**	**0**	**0**

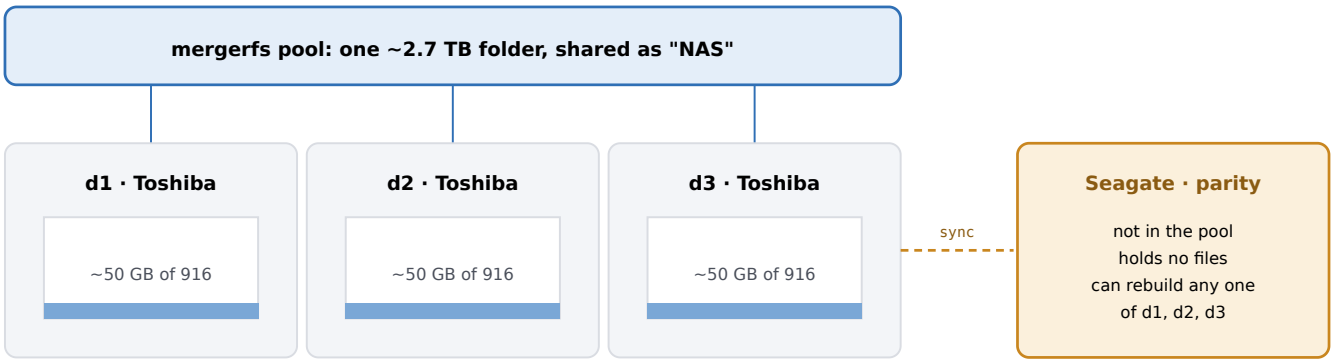
```
# Compare-only check: lists anything missing or different, copies nothing
robocopy "D:\Photos" "F:\NAS Photos\Photos" /E /L /NJH /NDL /FP
```

The GoPro folder was checked by comparing every file name and size directly. The other two folders were large enough that robocopy's list-only mode (`/L`) was the reliable way to check every file.

mergerfs + SnapRAID: one big drive that survives a failure

mergerfs joins several drives into one folder tree without striping. Each file lives whole on a single drive, so if a drive dies, files on the others stay readable. The *most free space* (mfs) create policy sends each new file to the emptiest drive, which is why all four drives held the same 38 GB.

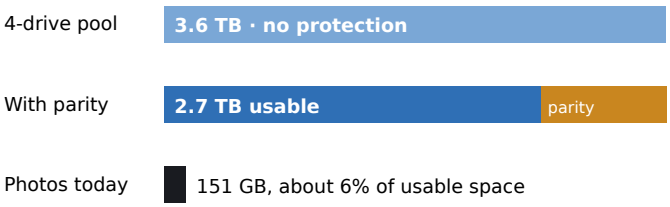
SnapRAID adds protection. One drive stores parity, a mathematical summary of the data drives. If any single data drive fails, its contents can be rebuilt from the other two plus parity. Unlike traditional RAID, it works on drives that already have files, and it runs on a schedule rather than in real time. The first sync read ~150 GB at about 39 MB/s and took roughly an hour, a normal speed for 2.5-inch USB drives.



Content files (SnapRAID's index of every file) are stored on all three data drives, so losing one never loses the index.

Fig. 7 · Final layout: three data drives in the pool, one parity drive outside it.

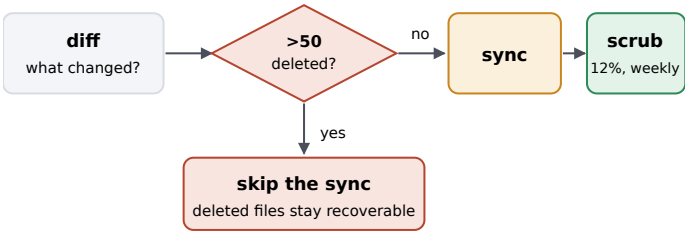
USABLE CAPACITY (TB)



One drive (~0.9 TB) buys recovery from any single failure.

Fig. 8 · The trade-off in space.

EVERY NIGHT (PLUGIN DIFF SCRIPT)



Pre-hash is on: new data is checksummed before parity is built.

Fig. 9 · The nightly job and its safety check.

SNAPRAID SETTING	VALUE	REASON
Parity	Seagate, parity level 1	Survives one drive failure at a time
Data + content	d1, d2, d3 (the Toshibas)	Index stored on all three
Schedule	Plugin diff script, daily	Protection is at most a day behind
Delete threshold	50 files	A mass deletion pauses syncing so it can be undone
Scrub	Every 7 days, 12% per run	Every file re-checked for silent corruption about every two months

Turning a full drive into the parity drive

SnapRAID was added after the photos were already pooled across all four drives, so one drive had to be emptied without taking the photos offline. Because mergerfs keeps each file whole on one drive, this is a file move, not a rebuild.

1. **Stop Immich** (Compose > Down) so nothing writes to the pool mid-move.
2. **Remove the Seagate's path** from the pool. Its 38 GB of files temporarily drop out of view but stay on the drive.
3. **Copy them back into the pool**, which places them on the remaining three drives:

```
rsync -aHAX --info=progress2 --exclude='lost+found' --exclude='aquota.*' \
  /srv/dev-disk-by-uuid-<seagate>/  /srv/mergerfs/pool/
# 40,410,532,813 bytes  100%
```

4. **Verify byte for byte** with a checksum dry run. No file names printed means every file matched:

```
rsync -aHAXn --checksum --out-format='%n' <seagate>/ /srv/mergerfs/pool/ | grep -v '/$'
```

5. **Clear the Seagate**, leaving it empty at 2.1 MB used. The pool still showed 151 GB.
6. **Assign drives in the SnapRAID plugin**, then run the first sync.

> **The hiccup:** while editing the pool, the Seagate's path was replaced by the pool's own shared folder, `/srv/mergerfs/pool/NAS/`, instead of being deleted. A pool that contains itself loops forever, so `df` and even a remount hung. Reading the unit file without touching the mount found it: `grep -o 'branches=[^ ]*' /etc/systemd/system/srv-mergerfs-pool.mount`. Removing the bad path and rebooting fixed it. Wrapping checks in `timeout 20` kept later commands from freezing the terminal.

Sharing the pool on the home network

Setup

1. Create a user account for file access, separate from the dashboard admin.
2. Create a shared folder `NAS` on the mergerfs pool, not on a single drive.
3. Grant that user read/write on the folder.
4. Enable SMB/CIFS and add the folder as a share.
5. In Windows, **Map network drive** to the share, with "Reconnect at sign-in" on.

Phones can open the same share through the iPhone Files app (Connect to Server) or an Android SMB file manager.

Three logins, three jobs

LOGIN	USED FOR
Dashboard admin	Managing the NAS in a browser
NAS user	The mapped drive and Files apps
Immich account	The photo app (one per person)

Immich keeps its own accounts and doesn't use OpenMediaVault's users. Family members who only use the photo app only need an Immich account.

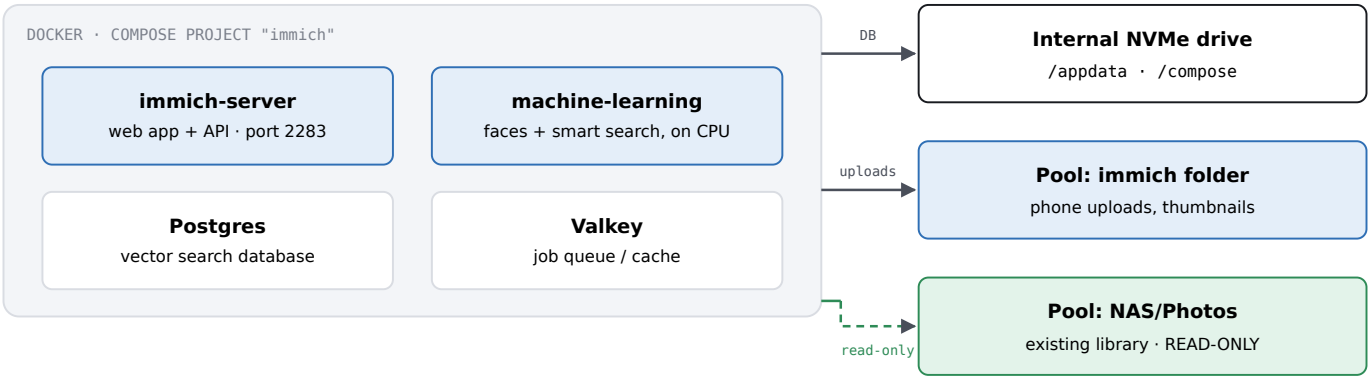
> **Why file sharing wouldn't start:** `smbd` showed *inactive (dead)*. Reading the saved configuration showed the share enabled but the service itself disabled. The settings page had been silently refusing to save, because "share user home directories" was switched on while home directories weren't set up. A 500 error revealed it. Switching that one option off let the page save.

```
omv-confdbadm read conf.service.smb | python3 -m json.tool | grep -B2 -A2 '"enable"'
# "enable": false   <- service switch, the real problem
# "enable": true    <- the NAS share itself
```

10 · PHOTO APP

Immich: Google Photos on hardware you own

Immich adds a timeline, albums, face grouping, a map, smart search ("beach", "dog") and automatic phone backup. It runs as four Docker containers defined in a Compose file, managed from OpenMediaVault's Compose plugin.



The database lives on the internal SSD because Immich's database isn't reliable on FUSE/pooled storage.
The original photos are mounted read-only as an "external library": Immich can index them but never alter or delete them.

Fig. 10 · Immich's containers and where each one stores data.

Setup path and problems solved

STEP	NOTE
Install Compose + sharerootfs plugins	sharerootfs allows shared folders on the system drive
Create `compose` and `appdata` folders	The plugin requires the compose-files and data folders to be different
Use Immich's official compose file	Added a read-only mount of the photo library at `/mnt/photos`
Environment variables	First attempt failed ("variable is not set"): the paste had missed the Environment box, leaving the generated `.env` empty
Pull images	ghcr.io returned `toomanyrequests`; pulling one image at a time with a retry loop worked
Create admin, add external library	Path had to be typed by hand; a pasted line had picked up hidden characters

```
until docker pull ghcr.io/immich-app/immich-machine-learning:v3; do echo "retrying in 60s..."; sleep 60; done
```

11 · REMOTE ACCESS

Reaching the NAS from anywhere, without opening ports

The usual shortcut, port forwarding on the router, puts the dashboard and file shares directly on the internet, where they get scanned and attacked constantly. Tailscale avoids that. Each device joins a private, encrypted network tied to one account, and they connect to each other directly.

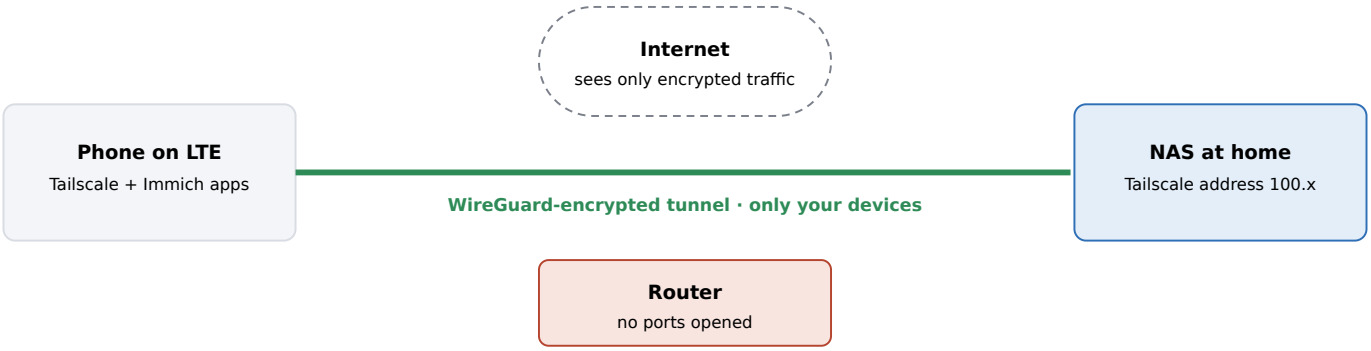


Fig. 11 · Away from home, the phone reaches Immich and the shares through the tunnel, as if it were on the home Wi-Fi.

```
wget -qO- https://tailscale.com/install.sh | sh    # curl isn't installed on OMV; wget is
tailscale up                                       # prints a login link to approve the device
```

- **Key expiry disabled** for the NAS in the admin console, so it never silently drops off the network.
- **Cost:** the Personal plan is free, for up to 6 users with unlimited devices.
- **Main risk:** whoever controls the sign-in account controls the network, so that account needs two-step verification. A lost phone gets removed from the admin console.
- **Immich app** pointed at the NAS's Tailscale address works both at home and away.

What went wrong, and how each problem was found

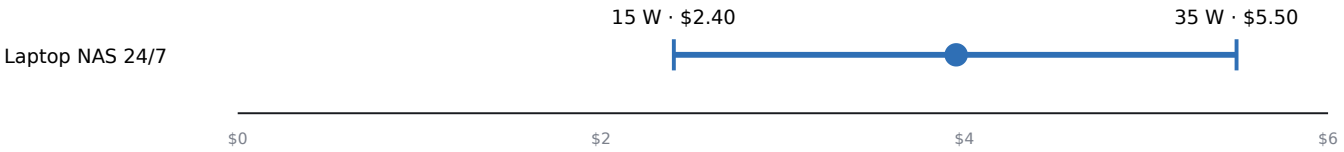
SYMPTOM	CAUSE	FIX	DIAGNOSED WITH
Updates "lost connection"	The dashboard restarts itself during its own update	Wait, refresh, re-check updates; apply pending changes	Update list
Router showed laptop as "msi" at a different address	Stale Windows Wi-Fi entry; the NAS uses the Ethernet adapter's MAC	Reserved the address by MAC on the router admin page	Network > Interfaces MAC
Only 1 of 4 file systems listed, stuck on "Creating"	Page refresh during slow full-disk ext4 formatting; 3 drives formatted but never mounted	Mounted the three, formatted the fourth one at a time, left the page alone	`ps aux | grep mkfs`, `lsblk -f`
Windows "can't connect" to share	SMB service disabled; settings page wouldn't save (home directories option)	Turned off home-directory sharing, saved, applied	`systemctl status smbd`, `omv-confdbadm`
robocopy "path not found"	Ran inside SSH first; then a mapped-drive / OneDrive Desktop log path	Used File Explorer copy instead	Prompt type, error text
Immich: "variable is not set"	Environment box empty, so `.env` was blank	Re-entered variables in the Environment field	`ls -la`, `cat immich.env`
`toomanyrequests` pulling images	Registry throttling parallel pulls	Sequential pulls in a retry loop	Pull output
Pool commands hang; remount fails	Pool included its own folder as a branch (self-loop)	Removed the bad branch, rebooted	Unit file read with `grep`, `timeout 20 df`
"curl: command not found"	Minimal Debian install	Used `wget -qO-` for the Tailscale installer	Shell error
No battery charge limit	Not offered by MSI software on this model; `msi-ec` unsupported	Accepted; periodic swelling check	`modprobe msi-ec`

> A pattern ran through several of these: **the web dashboard reported success while the underlying service hadn't changed.** Checking the actual system state over SSH (`lsblk`, `systemctl`, the saved config) found the real problem every time.

What it costs to keep running

Estimated from typical idle draw, not yet measured with a meter. An idle laptop with the lid closed and the CPU on powersave draws roughly 10–20 W, the four drives and hub add a few watts, and the RTX 3060 may not fully power down under Linux, which pushes toward the high end.

ESTIMATED MONTHLY COST AT ~22¢/kWh (MARYLAND AVERAGE, OCT 2026)



About $35–65 a year, roughly 1–2% of an average Maryland household bill. A plug-in power meter would replace the estimate with a measurement.

Fig. 12 · The range is the uncertainty in idle draw, mostly the graphics chip. The dot marks the midpoint.

Network impact

ACTIVITY	USES INTERNET BANDWIDTH?	EFFECT
NAS idle	Only for updates	None; gaming and streaming unaffected
Copying files at home	No, stays on the home network	Large Wi-Fi copies can briefly crowd other Wi-Fi devices
Phone backup at home	No	First full backup is large; run overnight
Remote access via Tailscale	Yes, home upload	Heavy remote streaming uses home upload while it runs

What I'd tell myself at the start

1. **A NAS is not a backup.** It's where the photos live. Parity protects against a dead drive, not against fire, theft or a mistake that wipes everything. A second copy somewhere else is still on the list.
2. **Verify before anything irreversible.** Robocopy's list-only mode turned "I think it copied" into "10,115 files, 0 mismatched" in seconds.
3. **Trust the system, not the page.** Several dashboard actions looked successful while the service underneath hadn't changed. `lsblk`, `systemctl` and reading the saved config were the source of truth.
4. **Read config without touching what's broken.** When the pool hung, reading its unit file from a second SSH session found the cause without making things worse.
5. **One slow job at a time.** Formatting and image pulls both failed when run in parallel or interrupted; one at a time just worked.
6. **Cheap hardware decisions compound.** Reusing the laptop skipped about $258 of Pi parts (board, power supply, cooler, SD card, screws) for the price of a replacement charger, and gave the headroom to run a full photo app with machine learning on the same box.

Next

ITEM	STATUS
Off-site or second copy of the photos (cloud or an external drive kept elsewhere)	RECOMMENDED
Pi-hole network ad blocker: on this NAS in Docker, or on its own Raspberry Pi	PLANNED
Family Immich accounts and partner sharing	PLANNED
Measure real power draw; try powering down the unused RTX 3060	OPTIONAL
Email alerts for drive health and full drives	OPTIONAL

> **Final state, Nov 1, 2026:** OpenMediaVault 8.3.1 on Debian 13 · 3 × 1TB data drives pooled with mergerfs (2.7 TB) · 1 × 1TB SnapRAID parity with nightly sync and weekly scrub · SMB share · Immich v3 in Docker · Tailscale remote access · lid-closed 24/7 operation.